PROJECT 3 · JUDGE THE JUDGE

AI judges wave through replies that promise too much

Customer: “Can I keep the car a few days longer?”

> A: You can usually extend your rental in the app or by phone, **as long as the car is not reserved for another customer.**

> B: You can usually extend your rental in the app or by phone. ~~[condition dropped]~~

The starting judge passes both. B commits SIXT to something the policy does not guarantee, and the judge catches only **7%** of dropped conditions on dev. A release gate built on it lets these replies ship.

INSIGHT

A judge that reads one reply cannot see what is missing.

So we make the absence explicit.

For every claim, call 1 lists the conditions and hedges the **policy** attaches to it and marks each one *kept*, *dropped* or *not applicable*. A dropped condition becomes a visible field instead of a gap.

0.067 → 0.667 / 0.633

dropped-condition catch rate, baseline → ours (dev, grouped 3-fold CV, two runs)

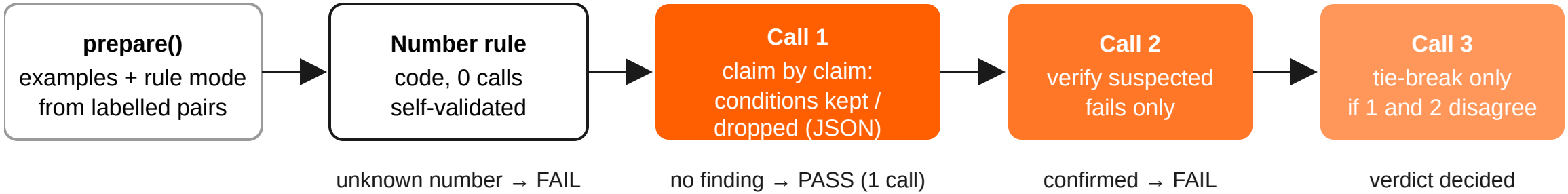
The rest is generic: certainty on a 0–3 scale (“usually” vs “always”), scope (some vs all branches), and numbers checked in code. Nothing about the criterion is hard-coded; examples and rule mode come from the labelled pairs.

Numbers from score.py reports: starter_pack/baseline_report.txt, results/*_report.txt

METHOD

Code for facts, the LLM for meaning, code for the verdict

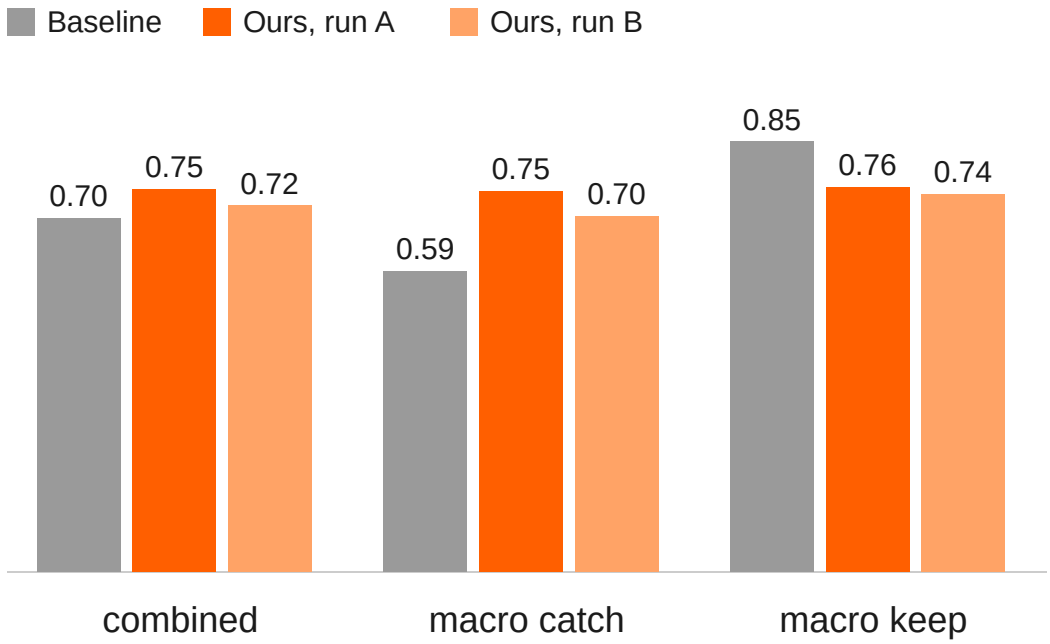
verdict computed in code · one reply per call · no state · ≤ 3 calls · claude-haiku-4.5, temperature 0



- **Self-validating rule:** a number not in the policy fails the reply, only if `prepare()` sees it never fire on a pass example of the current criterion; otherwise hint or off.
- **Verify only suspected fails** in a fresh context; a third call only when calls 1 and 2 disagree. 1.73 calls per reply on average.
- Same code for the surprise criterion: only `--criterion` and `--dev` change.

RESULTS · DEV, GROUPED CROSS-VALIDATION, claude-haiku-4.5

More bad replies caught; good replies still cost us



crit.	judge	combined	catch	keep	kappa	repeat
1	Baseline	0.697	0.592	0.847	0.531	n/a
1	Ours, run A	0.754	0.750	0.758	0.623	0.974
1	Ours, run B	0.721	0.700	0.744	0.575	0.974
2	*runs on the new criterion at 14:30, code unchanged*					

What did not work: keep fell below the baseline: the judge fails some correct replies (correct original passed: 0.805 / 0.765 vs baseline 0.858), and each failed original costs every pair built from it. Evidence-quoting variants scored lower on subsets. Identical runs differ by up to 0.033 combined, so a change ships only if it wins in both repeats. “repeat” = same verdict on two runs.

Sources: starter_pack/baseline_report.txt, results/exp_base_dev_dev_pairs_report.txt, results/dev_pairs_report.txt

FOR SIXT

A release gate for chatbot changes, with the reason attached

- Run every candidate chatbot version against the policy before release; a reply that drops a condition or raises certainty is blocked.
- Each fail carries the quoted reply words and the policy clause, so a reviewer decides in seconds.
- The recipe takes any new criterion plus a few labelled examples, with no code change (shown live on the surprise criterion).

Next steps

- Cut false fails on correct replies: call 2 must quote the violating words and name a concrete case (switch `VERIFY_WITNESS`, ready to measure).
- Retrieve the closest labelled examples per reply instead of a fixed set.
- Measure transfer on edit types not in dev (leave-one-edit-type-out).